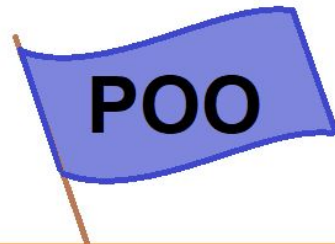


POO

By: Guillermo Andres De Mendoza Corrales





Abstracción

Encapsulamiento

Herencia

Polimorfismo

1. Abstraccion

Recrear una idea en memoria



```
class Perro:
```

```
    def __init__(self,nombre,raza,edad,color,altura,peso):  
        self.nombre=nombre  
        self.raza=raza  
        self.edad=edad  
        self.color=color  
        self.altura=altura  
        self.peso=peso
```



```
p1 = Perro("rex","pastor aleman",8,"negro",60,50)
```



```
Perro("mateo","french poodle",1,"blanco",30,20)
```

```
class Perro:

    def __init__(self,nombre,raza,edad,color,altura,peso):
        self.nombre=nombre
        self.raza=raza
        self.edad=edad
        self.color=color
        self.altura=altura
        self.peso=peso

    def saludar(self):
        print(f"wauiuuuuuu soy {self.nombre} un {self.raza}")
```

(Metodos)

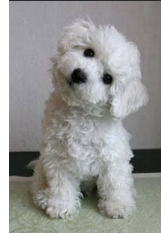


```
p1 = Perro("rex","pastor aleman",8,"negro",60,50)
p1.saludar()
```

wauiuuuuuu soy rex un pastor aleman

```
p2 = Perro("mateo","french poodle",1,"blanco",30,20)
p2.saludar()
```

wauiuuuuuu soy mateo un french poodle



2. Encapsulamiento

Proteger las variables de los objetos

```
class Perro:

    def __init__(self,nombre):
        self.nombre=nombre

    def saludar(self):
        print(f"waauuuuuu soy {self.nombre} un {self.raza}")

    @property
    def nombre(self):
        return self.__nombre

    @nombre.setter
    def nombre(self,nombre):
        if not isinstance(nombre, str):
            raise Exception("La variable debe ser string")
        if len(nombre) < 3:
            raise Exception("La variable debe tener minimo 3 caracteres")
        self.__nombre=nombre
```

```
p1 = Perro("rex")
```

```
p1 = Perro(1)
```

```
-----  
Exception                                 Traceback (most recent call last)  
/tmp/ipykernel_1704/407642502.py in <cell line: 0>()  
----> 1 p1 = Perro(1)
```

⬆ 1 frames

```
/tmp/ipykernel_1704/3404725035.py in nombre(self, nombre)  
14 def nombre(self,nombre):  
15     if not isinstance(nombre, str):  
---> 16         raise Exception("La variable debe ser string")  
17     if len(nombre) < 3:  
18         raise Exception("La variable debe tener minimo 3 caracteres")
```

Exception: La variable debe ser string

```
p1 = Perro("re")
```

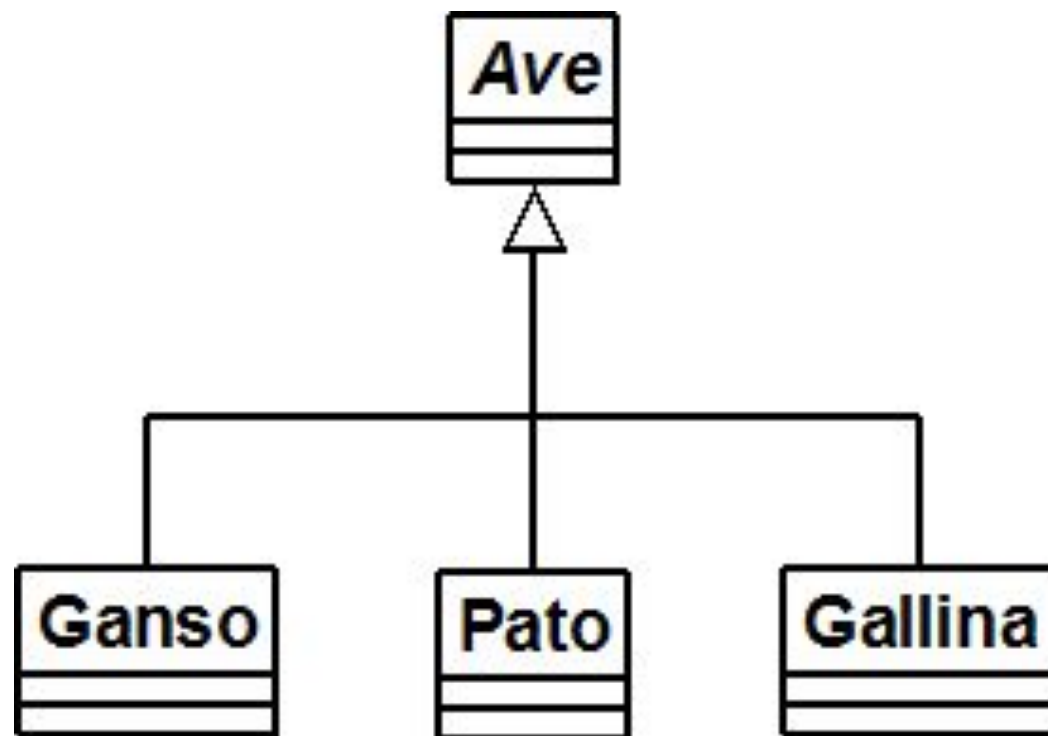
```
-----  
Exception                                 Traceback (most recent call last)  
/tmp/ipykernel_1704/1313490461.py in <cell line: 0>()  
----> 1 p1 = Perro("re")
```

⬆ 1 frames

```
/tmp/ipykernel_1704/3404725035.py in nombre(self, nombre)  
16         raise Exception("La variable debe ser string")  
17     if len(nombre) < 3:  
---> 18         raise Exception("La variable debe tener minimo 3 caracteres")  
19     self.__nombre=nombre
```

Exception: La variable debe tener minimo 3 caracteres

3. Herencia



```
class Mascota:

    def __init__(self,nombre):
        self.nombre = nombre

    def respirar(self):
        print("Respirando como Mascota")
```



```
class Perro(Mascota):

    def __init__(self,nombre,raza):
        super().__init__(nombre)
        self.raza=raza

    def saludar(self):
        print(f"waauuuuuu soy {self.nombre} un {self.raza}")
```



```
class Gato(Mascota):

    def __init__(self,nombre,raza):
        super().__init__(nombre)
        self.raza=raza

    def saludar(self):
        print(f"miauuuuu soy {self.nombre} un {self.raza}")
```

```
p1 = Perro("rex","pastor aleman")  
p1.saludar()  
p1.respirar()
```

wauuuuuuu soy rex un pastor aleman
Respirando como Mascota

```
g1 = Gato("michi","pardo")  
g1.saludar()  
g1.respirar()
```

miauuuuuu soy michi un pardo
Respirando como Mascota

4. Polimorfismo

```
p1 = Perro("rex","pastor aleman")
g1 = Gato("michi","pardo")
p2 = Perro("pancho","golden retriever")
g2 = Gato("gartfield","birmao")

mascotas = [p1,g1,p2,g2]
for mascota in mascotas:
    mascota.saludar()

wauuuuuuu soy rex un pastor aleman
miauuuuuu soy michi un pardo
wauuuuuuu soy pancho un golden retriever
miauuuuuu soy gartfield un birmao
```

5. Metodos especiales: str

```
class Person:

    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def __str__(self):
        return f"{self.nombre} tiene {self.edad} años"
```

```
p1 = Person("guillermo", 37)
print(p1)
```

```
guillermo tiene 37 años
```

5. Metodos especiales: repr

```
class Person:

    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def __repr__(self):
        return f"{self.nombre} tiene {self.edad} años"
```

```
p1 = Person("guillermo", 37)
p2 = Person("mario", 50)
p3 = Person("luigi", 45)
personas = [p1,p2,p3]
print(personas)
```

```
[guillermo tiene 37 años, mario tiene 50 años, luigi tiene 45 años]
```

5. Metodos especiales: eq

```
class Person:

    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def __eq__(self, other):
        return self.nombre == other.nombre and self.edad == other.edad
```

```
p1 = Person("guillermo", 37)
p2 = Person("luigi", 45)
p3 = Person("guillermo", 37)
```

p1 == p2

False

p1 == p3

True

5. Metodos especiales: lt y gt

```
class Person:
```

```
    def __init__(self, nombre, edad):
```

```
        self.nombre = nombre
```

```
        self.edad = edad
```

```
    def __lt__(self, other):
```

```
        if self.nombre == other.nombre:
```

```
            return self.edad < other.edad
```

```
        return self.nombre < other.nombre
```

```
    def __repr__(self):
```

```
        return f"[{self.nombre}:{self.edad}]"
```

```
p1 = Person("guillermo", 37)
```

```
p2 = Person("boo", 45)
```

```
p3 = Person("luigi", 37)
```

```
p4 = Person("warrio", 60)
```

```
p5 = Person("peach", 25)
```

```
personas = [p1,p2,p3,p4,p5]
```

```
personas.sort()
```

```
print(personas)
```

```
[[boo:45], [guillermo:37], [luigi:37], [peach:25], [warrio:60]]
```


5. Metodos especiales: `__len__`

```
class Box:  
  
    def __init__(self):  
        self.items = []  
  
    def __len__(self):  
        return len(self.items)
```

```
box = Box()  
box.items.append("item 1")  
box.items.append("item 2")  
box.items.append("item 3")
```

```
len(box)
```

```
3
```

5. Metodos especiales: `__getitem__`

```
class Box:

    def __init__(self):
        self.items = []

    def __getitem__(self, i):
        return self.items[i]
```

```
box = Box()
box.items.append("item 1")
box.items.append("item 2")
box.items.append("item 3")
```

```
print( box[1] )
```

```
item 2
```

5. Metodos especiales: `__contains__`

```
class Box:

    def __init__(self):
        self.items = []

    def __contains__(self, e):
        return e in self.items
```

```
box = Box()
box.items.append(1)
box.items.append(2)
box.items.append(3)
```

```
print(2 in box)
print(4 in box)
```

```
True
False
```

5. Metodos especiales: `__iter__`

```
class Box:

    def __init__(self):
        self.items = []

    def __iter__(self):
        for item in self.items:
            yield item
```

```
box = Box()
box.items.append("item 1")
box.items.append("item 2")
box.items.append("item 3")
```

```
for e in box:
    print(e)
```

```
item 1
item 2
item 3
```